Türkan Begüm Ahmetoğlu
31321

Report of PA1

Process Hierarchy During Pipe Command Execution:
In the executeCmd function, when a CMD_PIPE node is encountered, the parent shell process creates all child processes sequentially in a single for loop. So process 0 is forked first, then process 1, then process 2, etc. The parent creates them one after another. This is the order of process creation.

They can run concurrently to a significant extent. Once a child is forked, it begins executing independently. The parent does not call wait() inside the loop, so it immediately forks the next child. This means by the time the last child is forked, the earlier children are already running. The parent only waits for all of them after the loop. The children overlap in execution. This is how pipelining achieves parallelism.

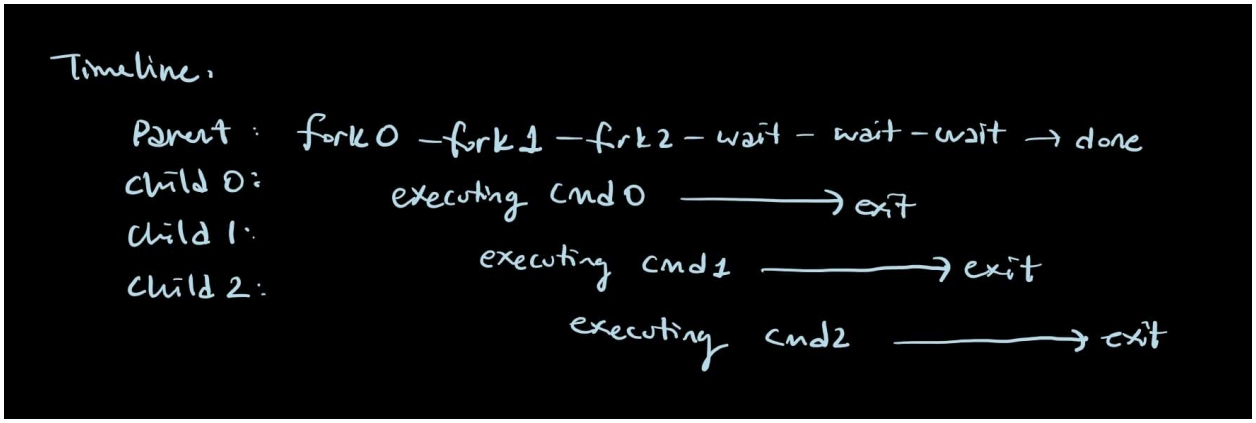


Figure 1: The timeline of the execution

You can see the timeline of the execution from the figure (Figure 1). In the beginning, the parent moves quickly through all three fork() calls which are represented by the short fork0, fork1, fork2 blocks on the parent's line. Each child starts running as soon as it is forked, shown by its bar beginning right after its corresponding fork() on the parent line. Because the parent doesn't wait between forks, Child 0, Child 1, and Child 2 overlap in execution. The parent then enters its wait() loop. It blocks until each child exits, one by one, before the shell is ready again. The staggered exits of the children reflect the fact that each command may take a different amount of time. Data flows left to right through the pipes connecting them, so in practice Child 1 cannot finish before Child 0 has produced its output, and Child 2 cannot finish before Child 1.

Pipe Structure Between Piped Processes:

For a command with n piped processes, there are n-1 pipes which gives one pipe between each consecutive pair of commands.

```
int pipes[n - 1][2]; // n-1 pipes each with read and write end

// create pipes
for(int i = 0 ; i < n-1 ; i++){

    if (pipe(pipes[i]) == -1) {
        perror("pipe failed");
        exit(1);
    }
}
```

Figure 2

Each pipe is a pair of file descriptors: pipes[i][0] is the read end and pipes[i][1] is the write end.

Inside the child process, I wire up the pipes using dup2():

```
// input from previous pipe
if (i > 0){
    dup2(pipes[i-1][0], STDIN_FILENO);
}

// output to next pipe
if (i < n-1){
    dup2(pipes[i][1], STDOUT_FILENO);
}
```

Figure 3

Every child except the first reads its stdin from the read end of the previous pipe (pipes[i-1][0]), and every child except the last writes its stdout to the write end of its own pipe (pipes[i][1]). This creates a clean chain, so each command's output becomes the next command's input. The first child keeps its original stdin (or uses input redirection if specified), and the last child keeps its original stdout (or uses output redirection if specified). All middle children communicate exclusively through pipes.

After forking each child, the parent closes the pipe ends it no longer needs:

```
if (i > 0) {
close(pipes[i - 1][0]);  // previous read end
}
if (i < n - 1) {
    close(pipes[i][1]);  // current write end
}
```

Figure 4

And inside each child, after wiring up its own stdin/stdout with dup2(), it closes all pipe file descriptors:

```
// close all pipes
for (int j = 0; j < n-1; j++){
    close(pipes[j][0]);
    close(pipes[j][1]);
}
```

Figure 5

This prevents deadlocks because a reading process blocks until the write end of its pipe is closed. If the parent or any other child holds the write end open, the reading child will block forever waiting for input that never arrives. By closing all unused ends immediately, I ensure that EOF is properly delivered to each child when the writer exits. Also it avoids file descriptor leaks because each process has a limited number of available file descriptors. Leaving unused pipe ends open can exhaust them. Closing them right after dup2() keeps the descriptor table clean throughout execution.

Executing a Conditional Command:
When a CMD_CONDITIONAL node is encountered, success or failure of the condition is determined by checking whether it produced any output on stderr. To capture this, a pipe dedicated to stderr is created first:

```
int error_pipe[2];
if (pipe(error_pipe) == -1) {
    perror("pipe failed");
    exit(1);
}
```

Figure 6

A child is then forked to execute the condition. Inside that child, stderr is redirected to the write end of the error pipe. This means any error output the condition produces goes into the pipe instead of the terminal. The child then calls executeCmd() recursively to run the condition, and exits when done.

Back in the parent, the write end of the error pipe is closed and the parent waits for the condition child to finish. Everything written to the pipe is then read:

```
// read all stderr output
char buffer[1024];
ssize_t total = 0, n;

while ((n = read(error_pipe[0], buffer + total, sizeof(buffer) - total)) > 0) {
    total += n;
}

close(error_pipe[0]);
```

Figure 7

So if anything was written to stderr (total > 0), the condition is considered to have failed, and the elseBranch is selected. If nothing was written (total == 0), the condition succeeded, and the thenBranch is selected. The captured stderr output is also replayed back to the terminal in the failure case, so the user still sees the error message. A second child is forked to execute whichever branch was selected, and the parent waits for it. This is how the chosen branch is executed.

Since the branch itself is a CmdNode, it can be of any type. When executeCmd(branch, cmd) is called on it, the entire process repeats: a new error pipe is created, a new condition child is forked, stderr is captured, and the appropriate sub-branch is selected. This recursion continues as deep as the nesting goes, with each level managing its own pipe and fork independently. This is how nested conditionals are handled recursively.

Input and Output Redirection:
Two dedicated functions handle redirection: handle_input_redirection() for stdin and handle_output_redirection() for stdout. Each checks whether a file path has been specified in the compiledCmd struct, opens the file, redirects the appropriate file descriptor using dup2(), and closes the original file descriptor.

In the conditional execution, the first forked child, the one running the condition, calls handle_input_redirection() before executing the condition. This ensures that if an input file was specified, the condition reads from it rather than from the terminal. Input redirection logically belongs to the condition because it is the first command in the pipeline.

Output redirection kept away from the condition and applied only to the correct branch by temporarily nulling out cmd->outFile before executing the condition, and restoring it afterwards:

```
char *saved_out = cmd->outFile;
cmd->outFile = NULL;

// execute condition
executeCmd(node->data.conditional.cond, cmd);

cmd->outFile = saved_out;
```

Figure 8

By setting cmd->outFile to NULL, any call to handle_output_redirection() inside the condition's execution will see no output file and do nothing. After the condition child exits, the parent restores the original value. When the second child is forked to run the chosen branch, it calls handle_output_redirection() with the restored cmd->outFile, ensuring the final output is correctly redirected.